Securing Orchestrated Multi-Agent Retrieval-Augmented Generation Systems Against Cascading Prompt Injection and Shared-Memory Poisoning

MHE 1 Option B - Revised Project for Approval

Prepared by: Shahiba Shamshad, Mark Paulraj, Sameer Khan

Project Overview

LLM apps are starting to use RAG and multiple agents together. One agent searches documents, another checks the results, another writes the answer or calls a tool. The problem is that retrieved documents can contain hidden instructions. If one agent reads a malicious document, it might follow those instructions instead of the user's actual request. This is called indirect prompt injection. Most research looks at this in single-agent RAG systems. We want to see what happens in a multi-agent setup. If one agent gets tricked, does the problem spread to the other agents? Does it get saved into shared memory? Can it reach an agent that has permission to call tools? We don't know yet.

Our project is to build a small multi-agent RAG system, attack it with prompt injection, and see how far the attack spreads. Then we'll try a few defenses and measure whether they actually help without breaking normal tasks.

Main Objective and Research Questions

Our main goal is to build a multi-agent RAG system, attack it with indirect prompt injection, and measure how far the attack spreads. Then we want to test some defenses and see if they actually help.

RQ1. Does a multi-agent RAG system get attacked more easily than a single-agent one? Or less easily? We don't know yet.
RQ2. If one agent reads a malicious document, does the problem stay with that agent, or does it spread to the others through the orchestrator?
RQ3. Does it matter if agents share memory? We'll compare three setups: no shared memory, shared read-only memory, and shared read/write memory.
RQ4. Does the type of retrieval change how vulnerable the system is? We'll try BM25, dense retrieval, and a mix of both.
RQ5. What happens if the attack reaches an agent that can call tools? Does that make things worse?
RQ6. Can we stop the attack with a combination of detection, tracking where content came from, and blocking suspicious memory writes? And does it hurt normal performance?
RQ7. If we tune our defense on some attacks, does it still work on new attacks we didn't train on?
RQ8. What's the cost of the defense? How many false positives, how much slower, how many extra tokens?

Proposed System and High-Level Architecture

We're planning four parts: an orchestrator and three agents.

The orchestrator takes the user's question, breaks it into steps, decides which agent goes next, and puts the final answer together.The retrieval agent searches the document store. It can use BM25, dense retrieval, or both. It returns the chunks it found along with where they came from. The security agent looks at what the retrieval agent found and decides if it's safe to use. It checks the detector score, the source, and whether the content should be accepted, quarantined, or thrown out. The action agent writes the final answer. If the task needs a tool call, it checks permissions first. There's also a shared memory store. In some experiments, agents can only read from it. In others, they can also write to it. We'll also build a logging layer that saves everything: what was retrieved, detector scores, messages between agents, memory

writes, tool calls, and final outputs. This way, when something goes wrong, we can see exactly where it happened instead of just knowing the final answer was bad.

Data Sources and Benchmarks:

We'll use two public benchmarks so our results can be compared to other work.

BIPIA is a benchmark for indirect prompt injection. It has tasks where malicious instructions are hidden in external content. We'll use it to build clean and attacked versions of the same RAG tasks.

AgentDojo is for testing prompt injection in agents that use tools. It has realistic tasks and known attacks. We'll use it for the part of our project where an agent can call tools.

We might also make a small set of our own test cases for things the benchmarks don't cover — like paraphrased attacks or attacks that try to persist in memory. If we do this, we'll keep them separate from the benchmark data and write down exactly how we made them.

For all the data, we'll keep track of where it came from, how many examples we used, and how we split it into train/validation/test. We'll also make sure we don't tune our detector on the same data we use to evaluate it.

Methodology

Baseline Systems: We'll compare against four baselines:

Baseline A is just a single-agent RAG system with no defense. Query goes in, retrieval happens, LLM answers.
Baseline B is our multi-agent system but with no defense. The agents talk to each other normally.
Baseline C uses defensive prompting. We tell the model in the system prompt that retrieved documents are untrusted and shouldn't override the user's request. This is a common first attempt at defense, but we don't expect it to work well.
Baseline D is a simple keyword filter. If a chunk contains phrases like "ignore previous instructions" or "system:", we flag it. This is deliberately basic — it's just there so we have something to compare our real defense against.

Retrieval Methods: We'll test three retrieval methods:

1. BM25 is keyword-based. It looks for exact word matches between the query and the documents.
2. Dense retrieval uses embeddings. It converts the query and documents into vectors and finds the closest matches by cosine similarity. This can find relevant content even when the words are different.
3. Hybrid combines both. We're not sure yet how we'll weight them — we might just take the top results from each and merge them.

We'll also try different chunk sizes and different top-k values to see if those change how vulnerable the system is. If we have time, we'll add a reranker step.

Advanced Method: Trust-Aware Multi-Agent RAG Defense

Our defense has four parts. We're calling it "trust-aware" because each piece tries to track how much we trust a given piece of content.

1. Injection detector. We'll fine-tune a small model to classify chunks as benign or injected. We're thinking DistilBERT or something similar. We have used LoRA before, so we might try that if the basic fine-tuning doesn't work well enough. We'll compare against just prompting a bigger model to classify.
2. Source tracking. Every chunk keeps track of where it came from. If a chunk came from an untrusted

document, that fact doesn't disappear just because an agent summarized it. We want to be able to trace back.
3. Trust score. We combine the detector score, the source, and the privilege level of the agent that's about to use the content into one number. Then we decide: allow it, verify it first, quarantine it, or block it.
4. Memory quarantine. If a chunk is suspicious, we don't let it get written into shared memory. It sits in quarantine until something verifies it.
5. Tool gate. If the answer agent wants to call a tool based on untrusted content, it needs an extra check before that happens. For the trust score, we're starting with something simple:

$$R = \alpha \times P_injection + \beta \times (1 - T_source) + \gamma \times P_privilege$$

where P_injection is the detector's score, T_source is how much we trust the source, and P_privilege is how risky the agent's permissions are. We'll set α, β, and γ experimentally — we don't know the right values yet. Same for the two thresholds between allow/verify/block. This is just a starting formula; we're not claiming it's theoretically optimal.

Scenario-Based Experimental Plan: We'll run a series of experiments. Each one adds something new so we can isolate what's causing what.

S0: Clean baseline. First we run everything with no attacks, just normal documents. This tells us what the system looks like when it's working. We measure retrieval quality and whether the answers are correct.
S1: Single-agent attack. We take one document, hide an injection in it, and feed it to a single-agent RAG system. How often does the attack work? How much worse are the answers?
S2: Multi-agent, no shared memory. Same attack, but now it's the multi-agent system. Each agent has its own memory. Does the attack stay contained, or does it still spread?
S3: Shared read-only memory. Now the agents share a document store but can't write to it. Does the shared access make the attack spread more?
S4: Shared read/write memory. Now agents can also write to shared memory. If an agent writes something based on the malicious document, does it persist even after the original document is gone?
S5: Privilege escalation. We put the malicious content where a low-privilege agent will find it. Does it reach the orchestrator? Does it reach the agent that can call tools?
S6: Seen vs. unseen attacks. We tune our defense on some attacks. Then we test it on paraphrased versions and attacks it hasn't seen. Does it still work?
S7: Retrieval sensitivity. We run the same attack with BM25, dense, and hybrid retrieval. We also vary top-k and chunk size. Does the choice of retrieval change how vulnerable we are?
S8: Persistence over time. We inject once, then run a bunch of clean tasks afterward. Does the poisoned memory still affect the system later?
S9: Defense comparison. We run the same attack set against: no defense, each defense component on its own, and all of them together. Which parts actually help?

Evaluation Metrics and Analytical Results

Retrieval quality. Did we actually find the right documents? We'll use Recall@k, MRR, and nDCG. These are standard for retrieval, and we're already using them in our coursework.
Detector performance. How well does our injection detector work? We'll look at precision, recall, F1, and the false positive and false negative rates. We care about false positives a lot — if the detector flags normal documents, the whole system becomes useless.
Security. How often does the attack succeed? How far does it spread between agents? How deep does it go? Does it stick around in memory? Does it cause bad tool calls? We'll track all of these separately so we

can see where things break down.
Utility. Does the system still work on normal tasks? We'll check whether benign tasks still succeed, whether answers are correct, and whether the system stays faithful to the retrieved content (i.e., doesn't make things up).
Cost. How much slower is the system with the defense on? How many extra tokens does it use? How much GPU time does fine-tuning take?

The important thing is that we report security and utility together. A defense that stops every attack but also blocks every normal request isn't useful. So we'll always show both numbers side by side.

Required Comparison, Ablation, and Failure Analysis

Performance / Baseline Comparison:

For the midterm, we'll compare three things: no defense, the simple keyword filter, and our trust-aware detector. We'll run all three on the same attack set so the numbers are comparable.
After the midterm, we'll add the full layered defense and the different retrieval methods.

Ablation Study

After the midterm, we'll test the full defense and then remove one piece at a time to see what each one is actually doing. We'll try:

- No detector
- No source tracking
- No memory quarantine
- No tool gate
- No trust score

This should tell us which parts are actually helping and which ones are just adding overhead. If something doesn't help, we'll drop it.

Structured Failure Analysis

When things go wrong, we want to know why. So we'll go through our results and pick out at least 20 cases where something failed in an interesting way. We'll group them into categories and describe at least five of them in detail.
The categories we're expecting:

- **Retrieval miss**: the right document wasn't found.
- **Detector missed it**: the attack got through because the detector thought it was benign.
- **False alarm**: a normal document got flagged as an attack.
- **Bad routing**: the orchestrator sent the task or the evidence to the wrong agent.
- **Spread between agents**: the attack moved from one agent to another.
- **Memory contamination**: bad content got written into shared memory.
- **Bad tool call**: the system called a tool when it shouldn't have.
- **Bad final answer**: the retrieved content was fine but the answer was wrong or made things up.

For each failure we look at, we'll write down: what type it was, how often it happened, one example, what probably caused it, what went wrong as a result, and what we could try to fix it.

Experimental Setup and Reproducibility

A few things we'll do to keep the project reproducible:

- Write down which version of BIPIA and AgentDojo we used and link to the exact source.

- Use the official train/validation/test splits if the benchmark has them. If not, we'll make our own and write down how.
- Run each stochastic experiment at least 3 times and report the average and standard deviation.
- Keep track of model names and versions, temperature, prompts, top-k, chunk size, embedding model, detector threshold, and any LoRA settings. Basically anything we change.
- Don't tune the detector on the same data we use for final evaluation.
- Use the same attack cases and settings when comparing defenses, so the comparison is fair.
- Keep everything in a Jupyter notebook or a GitHub repo, with the actual outputs saved, not just the final tables.

Three-Person Technical Work Division

Shahiba is doing the retrieval and memory side: building the RAG pipeline, setting up BM25, dense, and hybrid retrieval and testing the three memory setups (isolated, shared read-only, shared read-write). She'll also handle the retrieval quality metrics and the memory persistence experiments.
Mark is doing the multi-agent attack side: building the orchestrator, setting up how agents talk to each other, creating the injection scenarios, and measuring how far attacks spread and whether they reach agents with higher privileges.
Sameer is doing detection and defense :fine-tuning the injection detector, building the source tracking and trust score, and setting up the quarantine and tool gate. Then all three of us work together on integration, running the final comparison, and doing the error analysis. Each person owns a real piece of the system and has their own results to show.

MHE 1 / Midterm Scope (Option B): By the midterm, we'll have the final scope and problem statement done (this document). We'll have a working RAG pipeline and a basic orchestrator with 2–3 agents. BIPIA will be loaded and documented, with our evaluation protocol written down. We'll have clean vs attacked baseline results, at least two comparison points (no defense and the keyword filter), plus our first version of the trust-aware defense. We'll also have a results table with real numbers. For the failure analysis, we'll pick out 20 or more cases, group them into at least three categories, and write up five in detail. We'll do one depth analysis — probably single-agent vs multi-agent, or isolated vs shared memory. If the memory experiment isn't ready, we'll do retrieval and top-k sensitivity instead. We'll also have a high-level architecture diagram and a 3–5 minute demo showing one input, one result, one failure, and what's next.

Final Project Extension: After the midterm, we'll add the full shared read/write memory experiments and finish the LoRA or QLoRA detector if it isn't done yet. We'll also complete the full defense — source tracking, memory quarantine, and the tool gate. We'll test more attack types, including paraphrased, obfuscated, and unseen attacks. On the retrieval side, we'll run more tests with different chunk sizes and top-k values. We'll do the ablation study where we remove one defense piece at a time to see what each one is actually doing. We'll also measure cost — how much slower the system is with the defense on, how many extra tokens it uses, and how much GPU time fine-tuning takes. Finally, we'll expand the failure analysis to a larger sample.

Proposed Work Plan and Timeline

Right now we're finalizing the scope, the architecture, and the repo setup so we have a working environment. By the midterm, we want baselines, the orchestrator, attack cases, and a first version of the defense — that gives us code, a report, results, and a demo. After the first midterm phase, we'll add shared memory variants, the LoRA detector, and the full defense, which gives us working components and

scenario results. After the second phase, we'll do the ablation, persistence, and tool experiments and produce the final tables and failure analysis. In the final week, we'll do a reproducibility check and finish the write-up, report, code, and presentation.

Feasibility, Risks, and Fallback Plan

There are a few things that could go wrong.
Compute is the biggest one — fine-tuning multiple large models might not fit on Colab, so we'll use a small model with LoRA and skip the full multi-model comparison if we have to.
Scope is another risk — building the full multi-agent system with shared memory and tool security could get too big, so for the midterm we're focusing on one orchestrator setup, one benchmark, and one working defense, then adding the rest after.
Benchmark integration is a risk because BIPIA and AgentDojo use different task formats, so we'll use BIPIA for the main RAG work and AgentDojo only for a small tool-use extension.
False positives are a real concern — if the detector blocks normal documents, the system becomes useless, so we'll report false positives clearly and test different thresholds. Finally, **novelty** isn't our main goal — prompt injection already exists in prior work, so we're focusing on the comparison, the propagation measurements, and the integrated analysis rather than claiming the attack itself is new

Expected Contributions and Future Research Path

- A reproducible multi-agent RAG testbed using public benchmarks (BIPIA, maybe AgentDojo).
- A comparison of four setups: single-agent, isolated-memory multi-agent, shared read-only, and shared read/write.
- Numbers for how far attacks spread, how deep they go, and how long they stay in memory — these don't really exist yet for this kind of system.
- A layered defense: detector + source tracking + quarantine + tool gate.
- Full analysis: scenarios, ablation, security vs utility trade-offs, and structured failure cases.
- A foundation for later work on adaptive attacks, cross-model transfer, and stronger trust defenses.

References

1. Yi et al., "Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models," arXiv:2312.14197, 2023. (BIPIA)
2. Debenedetti et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," NeurIPS 2024.
3. Lee and Tiwari, "Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems," arXiv:2410.07283, 2024.
4. Wang et al., "G-Safeguard: A Topology-Guided Security Lens and Treatment on LLM-based Multi-agent Systems," ACL 2025.
5. Course notes from NLP class, 2026.